



OptiGrid

Energy Intelligence Platform

Coding Standards Document

Version 1.0

July 30, 2026

Team Coreflow

COS 301 - Capstone Project

University of Pretoria

Contents

Document Revision History	2
1 Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Intended Audience	3
2 Naming Conventions	3
2.1 Variables and Constants	3
2.2 Functions and Methods	3
2.3 Classes and Components	3
2.4 Files and Directories	3
2.5 Branches and Commits	3
3 Repository File Structure	4
4 Code Formatting and Style	5
5 Languages and Frameworks	5
6 Documentation and Commenting	5
6.1 Commenting Principles	5
6.2 API Documentation Standards	5
6.3 Supporting Documents	5
7 Tooling and Automated Enforcement	6
7.1 Linting Configuration	6
7.2 Type Checking Configuration	6
7.3 Continuous Integration Checks	6
8 Version Control Workflow	6
8.1 Branching Strategy	6
8.2 Pull Request Requirements	6
9 Quality Attributes	6

Document Revision History

Version	Date	Description of Change	Author
1.0	July 30, 2026	Initial version.	Team Coreflow

1 Introduction

1.1 Purpose

This standard works as a guide for the developers of the OptiGrid project. Its aim is to maintain a consistent coding strategy and style in the repository. It is a good reference that can be used by external personnel interested in understanding the technical work behind our platform.

1.2 Scope

The coding standards apply to majority of code in backend, frontend and testing. It also extends to the GitHub repository workflow and the deployment environment.

1.3 Intended Audience

This document is mainly intended for the CoreFlow team working on the OptiGrid project and the staff supervising the project. In addition to that, public contributors and reviewers may use it to help understanding the project.

2 Naming Conventions

2.1 Variables and Constants

- Normal Variables and Instances Constants: **camelCase**
- API Payloads/Database Schemas Variables: **snake_case**
- Configuration/Pattern Constants: **UPPER_SNAKE_CASE**

2.2 Functions and Methods

- TypeScript/JavaScript Functions: **camelCase**
- Python Functions: **snake_case**

2.3 Classes and Components

- TypeScript/Python Classes: **PascalCase**
- React Components: **PascalCase**

2.4 Files and Directories

- Files: **kebab-case** and **snake_case**
- Directories: **lowercase** and **kebab-case**

2.5 Branches and Commits

- Branches: **area/kebab-case-description**
- Commit Messages: **type(scope): description**

3 Repository File Structure

The structure of the project repository is organized as follows:

```

optigrid/
|-- .github
|   |-- workflows                % CI/CD pipeline definitions
|-- backend/                    % Server-side services
|   |-- core                    % Main API service (Node.js / Express)
|   |   |-- prisma              % Database schema and migrations
|   |   |-- src
|   |       |-- controllers
|   |       |-- services        % Business logic layer
|   |       |-- routes
|   |       |-- middleware      % Auth, RBAC, rate limiting
|   |       |-- validation      % Request schemas (Zod)
|   |       |-- lib
|   |       |-- app.ts
|   |       |-- server.ts
|   |-- ingestion               % Telemetry ingestion service (Python)
|   |-- analytics               % Forecasting service (Python)
|-- frontend/                  % Next.js web application
|   |-- app
|   |   |-- (auth)              % Login, signup, password reset
|   |   |-- (dashboard)         % Authenticated application pages
|   |   |-- api                 % Server-side API proxy routes
|   |-- lib
|   |-- styles
|   |-- __mocks__
|-- tests/                     % Test suites
|   |-- unit
|   |-- integration
|   |-- e2e                    % Playwright end-to-end tests
|-- infrastructure/            % Deployment configuration
|   |-- docker
|   |-- terraform
|-- docs/                      % Project documentation
|-- scripts/                   % Developer and deployment scripts
|-- supabase/                  % Database seed data
|-- docker-compose.yml
|-- eslint.config.cjs
|-- pnpm-workspace.yaml        % Monorepo workspace definition
|-- package.json
|-- README.md

```

4 Code Formatting and Style

- Indentation: **4 spaces**
- Maximum Line Length: **100 characters**
- Quotation: **Double quotes**
- Multi-line Blocks: **Curly braces**
- Equality: **Strict (===)**
- Constants Declaration: **Read-only (const)**
- Components Declaration: **Function declaration (export default function)**

5 Languages and Frameworks

- Programming Languages: **TypeScript/JavaScript, Python.**
- Frontend Framework: **Next.js (React)**
- Backend API: **Express**
- Data Services: **FastAPI**
- Database Access: **Prisma**
- Auth & DB Hosting: **Supabase**
- Database: **PostgreSQL**

6 Documentation and Commenting

6.1 Commenting Principles

Add comments throughout the code when necessary to explain code logic or indicate changes that need to follow.

6.2 API Documentation Standards

Use Swagger to document backend REST endpoints.

6.3 Supporting Documents

Include information about code configuration and running instructions in the README.

7 Tooling and Automated Enforcement

7.1 Linting Configuration

ESLint runs via flat config (`eslint.config.cjs/.mjs`) at the root, frontend, and backend, using `@typescript-eslint` to enforce coding standards.

7.2 Type Checking Configuration

TypeScript 5.8 is used on both sides. Backend checking is strict while frontend checking is not. The Type checking runs as part of each build script (`tsc` for the backend and `next build` for the frontend), and not as a separate typecheck step.

7.3 Continuous Integration Checks

For every pull request/commit to `develop/main`, GitHub Actions runs five jobs on Frontend, Backend Core, Python Service, Integration, and E2E to perform automated linting and testing.

8 Version Control Workflow

8.1 Branching Strategy

This project follows GitFlow. Use `infra` for configuration. Use `develop` branch as the primary developing area. Branch from `develop` to implement features. Merge `develop` to `main` before each version release. The `main` branch must always contain a working product.

8.2 Pull Request Requirements

- Must include sufficient description of the new features.
- Needs to pass all CI/CD checks.
- Should be reviewed and approved by at least 2 other team members.
- All conflicts with other branches should be resolved strategically.

9 Quality Attributes

The use of coding standards contribute to the following quality attributes:

Uniformity: The same style is followed by all developers to present a clean project.

Clarity: The code is readable and maintainable because of the use of industry coding conventions.

Flexibility: Standard templates allow easy features additions and changes to existing features.

Reliability: Errors and bugs are significantly reduced by implementing consistent patterns.

Efficiency: The development process becomes smoother due to increased predictability.
